Fitting Models to Data

Module 3: Sensitivity analysis and parameter estimation

Jim Faeder

MSCBIO 2025 · Introduction to Bioinformatics Programming in Python

Fall 2026

Resources

- **BioNetGen** software, documentation, and tutorials: bionetgen.org
- **VS Code extension**: search the Extensions panel for *BioNetGen*
- **BNG Playground**: write and simulate models in your browser, no installation ruleworld.github.io/bngplayground
- **bngsim**: fast ODE and stochastic simulation from Python bngsim.readthedocs.io · `pip install bngsim bionetgen`
- **Course repository**: models, notebooks, slides and handouts github.com/jrfaeder/RBM_Intro_BioNetGen



Course repository



BNG Playground

The book

Rule-Based Modeling of Biological Systems: Theory and practice with BioNetGen is an open book that develops the material of this course in more depth, with every model, figure, and exercise.

ruleworld.github.io/bionetgen-book

For this module, read:

- Chapter 7: Analyzing models (sensitivity analysis)
- Chapter 9: Parameter estimation (in preparation)



The book

Goals for this module

By the end of this module you should be able to

1. say where the parameters of a signaling model come from, and set **plausible ranges** when they are unknown

Goals for this module

By the end of this module you should be able to

1. say where the parameters of a signaling model come from, and set **plausible ranges** when they are unknown
2. convert between **concentrations and copy numbers**

Goals for this module

By the end of this module you should be able to

1. say where the parameters of a signaling model come from, and set **plausible ranges** when they are unknown
2. convert between **concentrations and copy numbers**
3. compute and interpret **local and global sensitivities**

Goals for this module

By the end of this module you should be able to

1. say where the parameters of a signaling model come from, and set **plausible ranges** when they are unknown
2. convert between **concentrations and copy numbers**
3. compute and interpret **local and global sensitivities**
4. explain the difference between a parameter that affects the outputs and one the data can determine (**identifiability**)

Goals for this module

By the end of this module you should be able to

1. say where the parameters of a signaling model come from, and set **plausible ranges** when they are unknown
2. convert between **concentrations and copy numbers**
3. compute and interpret **local and global sensitivities**
4. explain the difference between a parameter that affects the outputs and one the data can determine (**identifiability**)
5. fit a model by minimizing χ^2 with **multistart optimization**, and judge the estimates with **profile likelihoods**

Goals for this module

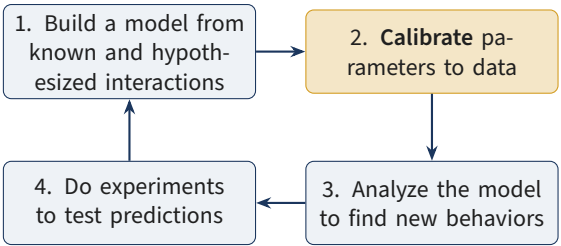
By the end of this module you should be able to

1. say where the parameters of a signaling model come from, and set **plausible ranges** when they are unknown
2. convert between **concentrations and copy numbers**
3. compute and interpret **local and global sensitivities**
4. explain the difference between a parameter that affects the outputs and one the data can determine (**identifiability**)
5. fit a model by minimizing χ^2 with **multistart optimization**, and judge the estimates with **profile likelihoods**
6. use the model to decide which **new experiment** would help

Part 1

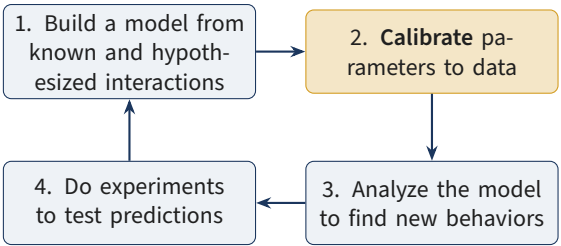
Where parameters come from

The modeling cycle



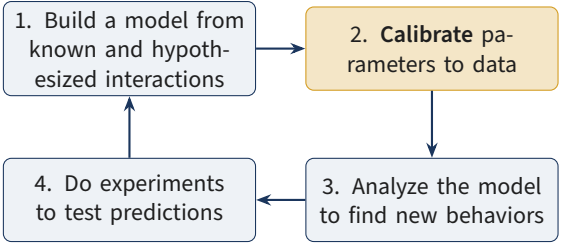
- Model behavior depends on its **structure** (the rules) and its **parameters**.

The modeling cycle



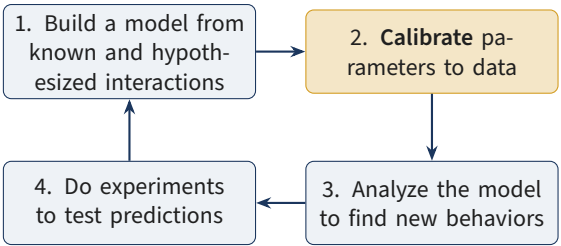
- Model behavior depends on its **structure** (the rules) and its **parameters**.
- Two kinds of parameters: **amounts** (copy numbers, concentrations) and **rate constants**.

The modeling cycle



- Model behavior depends on its **structure** (the rules) and its **parameters**.
- Two kinds of parameters: **amounts** (copy numbers, concentrations) and **rate constants**.
- Some can be measured directly. The rest must be **inferred from data**: find the parameters that make the model reproduce the measurements.

The modeling cycle



- Model behavior depends on its **structure** (the rules) and its **parameters**.
- Two kinds of parameters: **amounts** (copy numbers, concentrations) and **rate constants**.
- Some can be measured directly. The rest must be **inferred from data**: find the parameters that make the model reproduce the measurements.
- This module: how to tell which parameters matter, which ones data can determine, and how to estimate them.

Parameters are gleaned from many sources

kind	typical source
physical constants, volumes	textbooks, imaging
protein copy numbers	quantitative proteomics, flow cytometry, westerns
binding constants	SPR, ITC, other biophysics
catalytic rates	enzyme assays
everything else	**fitting to dynamic data**

Parameters are gleaned from many sources

kind	typical source
physical constants, volumes	textbooks, imaging
protein copy numbers	quantitative proteomics, flow cytometry, westerns
binding constants	SPR, ITC, other biophysics
catalytic rates	enzyme assays
everything else	**fitting to dynamic data**

In the 2003 model of FcεRI signaling, protein amounts came from antibody-based measurements, receptor binding kinetics from biophysical experiments, and the remaining rate constants from western blots of phosphorylation kinetics.

Parameters are gleaned from many sources

kind	typical source
physical constants, volumes	textbooks, imaging
protein copy numbers	quantitative proteomics, flow cytometry, westerns
binding constants	SPR, ITC, other biophysics
catalytic rates	enzyme assays
everything else	**fitting to dynamic data**

In the 2003 model of FcεRI signaling, protein amounts came from antibody-based measurements, receptor binding kinetics from biophysical experiments, and the remaining rate constants from western blots of phosphorylation kinetics.

Here we assume the **copy numbers are known** and estimate only the **rate constants**, a common situation now that proteomics gives copy numbers for most signaling proteins.

Faeder et al., *J. Immunol.* **170**, 3769 (2003)

Copy numbers: counting molecules per cell

File: `models/stat1_stat3.bngl`

- Concentration c (M) in volume V (L) $\rightarrow$ copy number $c\,N_A V$.

```
NA          6.022e23
Vcyt        1e-12          # L
cell_dens   1e9            # cells/L
Vext        1/cell_dens    # L per cell
L_conc      1e-9           # M
L0          L_conc*NA*Vext   # 6.0e5
S3_0        2e5            # 0.33 uM
kon         1e7            # 1/M/s
kon_m       kon/(NA*Vext) # 1.7e-8 /s
kS3on       1e6            # 1/M/s
kS3on_m     kS3on/(NA*Vcyt)  # 1.7e-6 /s
```

Rate constants stay in standard units, which is what we fit; the model converts them.

Copy numbers: counting molecules per cell

File: `models/stat1_stat3.bngl`

- Concentration c (M) in volume V (L) $\rightarrow$ copy number $c\,N_A V$.
- First-order rate constants (s^{-1}) need no conversion.

```
NA          6.022e23
Vcyt        1e-12          # L
cell_dens   1e9            # cells/L
Vext        1/cell_dens    # L per cell
L_conc      1e-9           # M
L0          L_conc*NA*Vext   # 6.0e5
S3_0        2e5            # 0.33 uM
kon         1e7            # 1/M/s
kon_m       kon/(NA*Vext)  # 1.7e-8 /s
kS3on       1e6            # 1/M/s
kS3on_m     kS3on/(NA*Vcyt)  # 1.7e-6 /s
```

Rate constants stay in standard units, which is what we fit; the model converts them.

Copy numbers: counting molecules per cell

File: `models/stat1_stat3.bngl`

- Concentration c (M) in volume V (L) $\rightarrow$ copy number $c\,N_A V$.
- First-order rate constants (s^{-1}) need no conversion.
- Second-order k ($M^{-1}s^{-1}$) $\rightarrow$ $k/(N_A V)$ per pair of molecules, where V is the volume in which they meet.

```
NA          6.022e23
Vcyt        1e-12          # L
cell_dens   1e9            # cells/L
Vext        1/cell_dens    # L per cell
L_conc      1e-9           # M
L0          L_conc*NA*Vext   # 6.0e5
S3_0        2e5            # 0.33 uM
kon         1e7            # 1/M/s
kon_m       kon/(NA*Vext)  # 1.7e-8 /s
kS3on       1e6            # 1/M/s
kS3on_m     kS3on/(NA*Vcyt)  # 1.7e-6 /s
```

Rate constants stay in standard units, which is what we fit; the model converts them.

Copy numbers: counting molecules per cell

File: `models/stat1_stat3.bngl`

- Concentration c (M) in volume V (L) $\rightarrow$ copy number $c\,N_A V$.
- First-order rate constants (s^{-1}) need no conversion.
- Second-order k ($M^{-1}s^{-1}$) $\rightarrow$ $k/(N_A V)$ per pair of molecules, where V is the volume in which they meet.
- Ligand meets receptors from the medium: $V_{ext} = 1/\rho_{cell}$, the medium volume per cell. Docking happens in the cytoplasm ($V_{cyt} \approx 1$ pL), promoter binding in the nucleus.

```
NA          6.022e23
Vcyt        1e-12           # L
cell_dens   1e9             # cells/L
Vext        1/cell_dens     # L per cell
L_conc      1e-9            # M
L0          L_conc*NA*Vext   # 6.0e5
S3_0        2e5             # 0.33 uM
kon         1e7             # 1/M/s
kon_m       kon/(NA*Vext) # 1.7e-8 /s
kS3on       1e6             # 1/M/s
kS3on_m     kS3on/(NA*Vcyt)  # 1.7e-6 /s
```

Rate constants stay in standard units, which is what we fit; the model converts them.

Copy numbers: counting molecules per cell

File: `models/stat1_stat3.bngl`

- Concentration c (M) in volume V (L) $\rightarrow$ copy number $c\,N_A V$.
- First-order rate constants (s^{-1}) need no conversion.
- Second-order k ($M^{-1}s^{-1}$) $\rightarrow$ $k/(N_A V)$ per pair of molecules, where V is the volume in which they meet.
- Ligand meets receptors from the medium: $V_{\text{ext}} = 1/\rho_{\text{cell}}$, the medium volume per cell. Docking happens in the cytoplasm ($V_{\text{cyt}} \approx 1$ pL), promoter binding in the nucleus.

Check: $k_{\text{on}}[L] = 10^{7} \times 10^{-9} = 10^{-2}\ s^{-1}$ in either set of units.

```
NA          6.022e23
Vcyt        1e-12           # L
cell_dens   1e9             # cells/L
Vext        1/cell_dens     # L per cell
L_conc      1e-9            # M
L0          L_conc*NA*Vext   # 6.0e5
S3_0        2e5             # 0.33 uM
kon         1e7             # 1/M/s
kon_m       kon/(NA*Vext) # 1.7e-8 /s
kS3on       1e6             # 1/M/s
kS3on_m     kS3on/(NA*Vcyt)  # 1.7e-6 /s
```

Rate constants stay in standard units, which is what we fit; the model converts them.

The model: STAT1 and STAT3 compete for the receptor

File: `models/stat1_stat3.bngl`

```
S1bind:   R(l!+,s) + STAT1(r,Y~0) -> R(l!+,s!1).STAT1(r!1,Y~0)            kS1on_m
S3bind:   R(l!+,s) + STAT3(r,Y~0) -> R(l!+,s!1).STAT3(r!1,Y~0)            kS3on_m
S3act:    R(l!+,socs,s!1).STAT3(r!1,Y~0) -> R(l!+,socs,s!1).STAT3(r!1,Y~P)  kact
Gbind:    STAT3(r,Y~P,g) + G(s) <-> STAT3(r,Y~P,g!1).G(s!1)               kGon_m, kGoff
SOCSbind: R(l!+,socs) + SOCS(r) -> R(l!+,socs!1).SOCS(r!1)                kSOCSon_m
```

- Built from Module 2, version 3, with a second STAT.

The model: STAT1 and STAT3 compete for the receptor

File: `models/stat1_stat3.bngl`

```
S1bind:   R(l!+,s) + STAT1(r,Y~0) -> R(l!+,s!1).STAT1(r!1,Y~0)            kS1on_m
S3bind:   R(l!+,s) + STAT3(r,Y~0) -> R(l!+,s!1).STAT3(r!1,Y~0)            kS3on_m
S3act:    R(l!+,socs,s!1).STAT3(r!1,Y~0) -> R(l!+,socs,s!1).STAT3(r!1,Y~P)  kact
Gbind:    STAT3(r,Y~P,g) + G(s) <-> STAT3(r,Y~P,g!1).G(s!1)               kGon_m, kGoff
SOCSbind: R(l!+,socs) + SOCS(r) -> R(l!+,socs!1).SOCS(r!1)                kSOCSon_m
```

- Built from Module 2, version 3, with a second STAT.
- STAT1 and STAT3 dock at the same receptor site, and the receptor kinase phosphorylates either one.

The model: STAT1 and STAT3 compete for the receptor

File: `models/stat1_stat3.bngl`

```
S1bind:   R(l!+,s) + STAT1(r,Y~0) -> R(l!+,s!1).STAT1(r!1,Y~0)            kS1on_m
S3bind:   R(l!+,s) + STAT3(r,Y~0) -> R(l!+,s!1).STAT3(r!1,Y~0)            kS3on_m
S3act:    R(l!+,socs,s!1).STAT3(r!1,Y~0) -> R(l!+,socs,s!1).STAT3(r!1,Y~P)  kact
Gbind:    STAT3(r,Y~P,g) + G(s) <-> STAT3(r,Y~P,g!1).G(s!1)               kGon_m, kGoff
SOCSbind: R(l!+,socs) + SOCS(r) -> R(l!+,socs!1).SOCS(r!1)                kSOCSon_m
```

- Built from Module 2, version 3, with a second STAT.
- STAT1 and STAT3 dock at the same receptor site, and the receptor kinase phosphorylates either one.
- Only pSTAT3 binds the *SOCS3* promoter, and SOCS3 blocks the kinase for both STATs: STAT3 drives the feedback that shuts off STAT1 too.

The model: STAT1 and STAT3 compete for the receptor

File: `models/stat1_stat3.bngl`

```
S1bind:   R(l!+,s) + STAT1(r,Y~0) -> R(l!+,s!1).STAT1(r!1,Y~0)            kS1on_m
S3bind:   R(l!+,s) + STAT3(r,Y~0) -> R(l!+,s!1).STAT3(r!1,Y~0)            kS3on_m
S3act:    R(l!+,socs,s!1).STAT3(r!1,Y~0) -> R(l!+,socs,s!1).STAT3(r!1,Y~P)  kact
Gbind:    STAT3(r,Y~P,g) + G(s) <-> STAT3(r,Y~P,g!1).G(s!1)               kGon_m, kGoff
SOCSbind: R(l!+,socs) + SOCS(r) -> R(l!+,socs!1).SOCS(r!1)                kSOCSon_m
```

- Built from Module 2, version 3, with a second STAT.
- STAT1 and STAT3 dock at the same receptor site, and the receptor kinase phosphorylates either one.
- Only pSTAT3 binds the *SOCS3* promoter, and SOCS3 blocks the kinase for both STATs: STAT3 drives the feedback that shuts off STAT1 too.
- Roughly the situation for IL-6, which signals mostly through STAT3.

The model: STAT1 and STAT3 compete for the receptor

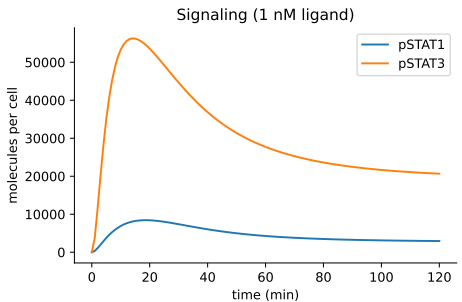
File: `models/stat1_stat3.bngl`

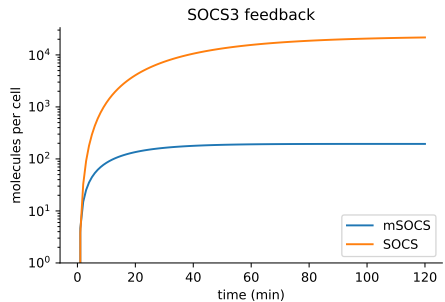
```
S1bind:   R(l!+,s) + STAT1(r,Y~0) -> R(l!+,s!1).STAT1(r!1,Y~0)            kS1on_m
S3bind:   R(l!+,s) + STAT3(r,Y~0) -> R(l!+,s!1).STAT3(r!1,Y~0)            kS3on_m
S3act:    R(l!+,socs,s!1).STAT3(r!1,Y~0) -> R(l!+,socs,s!1).STAT3(r!1,Y~P)  kact
Gbind:    STAT3(r,Y~P,g) + G(s) <-> STAT3(r,Y~P,g!1).G(s!1)               kGon_m, kGoff
SOCSbind: R(l!+,socs) + SOCS(r) -> R(l!+,socs!1).SOCS(r!1)                kSOCSon_m
```

- Built from Module 2, version 3, with a second STAT.
- STAT1 and STAT3 dock at the same receptor site, and the receptor kinase phosphorylates either one.
- Only pSTAT3 binds the *SOCS3* promoter, and SOCS3 blocks the kinase for both STATs: STAT3 drives the feedback that shuts off STAT1 too.
- Roughly the situation for IL-6, which signals mostly through STAT3.
- 18 rate constants, 29 species, 65 reactions.

Dynamics with the “true” parameters

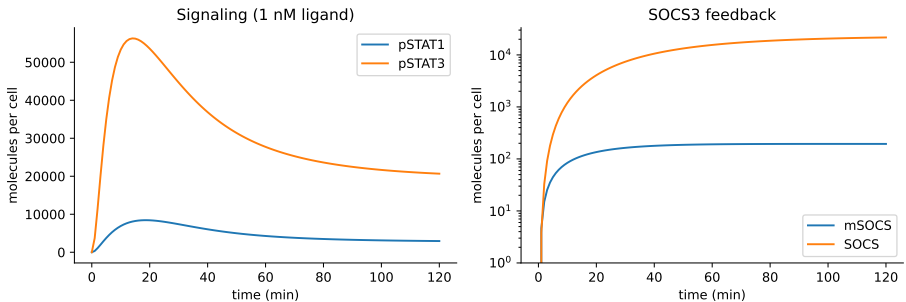
File: `models/stat1_stat3.bngl`





Dynamics with the “true” parameters

File: `models/stat1_stat3.bngl`



- pSTAT3 peaks at about 14 min, at 28% of total STAT3. pSTAT1 peaks at 8% of STAT1, because STAT3 wins most of the docking sites.
- SOCS3 reaches about 2×10^4 copies, more than the 5000 receptors, and holds both responses down.

Plausible ranges come from the class of process

Notebook: Section 1

Before fitting, say where each rate constant could lie, *without* looking at particular values:

class	parameters	range
association	`kon`, `kS1on`, `kS3on`, `kGon`, `kSOCSon`	10^{4}–10^{8} $M^{-1}s^{-1}$
dissociation	`koff`, `kS1off`, `kS3off`, `kGoff`, `kSOCSoff`	10^{-4}–10^{1} s^{-1}
catalysis, synthesis	`kact`, `kdiss`, `ktx`, `ktl`	10^{-3}–10^{1} s^{-1}
decay	`kS1dephos`, `kS3dephos`, `kmdeg`, `kSOCSdeg`	10^{-5}–10^{-2} s^{-1}

Plausible ranges come from the class of process

Notebook: Section 1

Before fitting, say where each rate constant could lie, *without* looking at particular values:

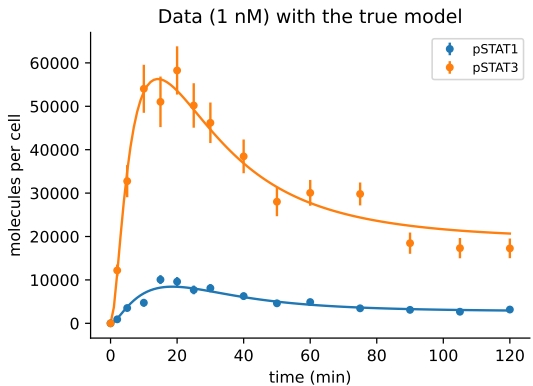
class	parameters	range
association	`kon`, `kS1on`, `kS3on`, `kGon`, `kSOCSon`	10^{4}–10^{8} $M^{-1}s^{-1}$
dissociation	`koff`, `kS1off`, `kS3off`, `kGoff`, `kSOCSoff`	10^{-4}–10^{1} s^{-1}
catalysis, synthesis	`kact`, `kdiss`, `ktx`, `ktl`	10^{-3}–10^{1} s^{-1}
decay	`kS1dephos`, `kS3dephos`, `kmdeg`, `kSOCSdeg`	10^{-5}–10^{-2} s^{-1}

- Association: slow protein binding up to near the diffusion limit. Decay: half-lives from a minute to a day.
- Work in $\log_{10} k$: a step of 0.1 is the same relative change at any scale.

Test the method on synthetic data

Notebook: Section 2

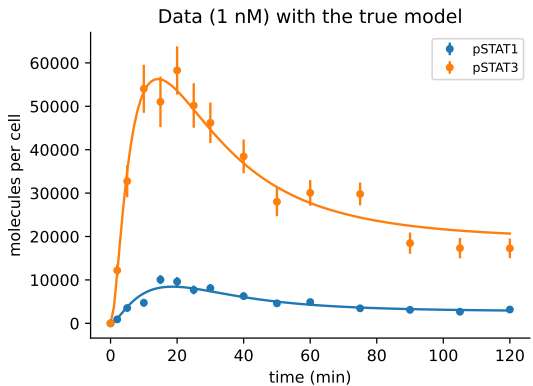
- Choose “true” rate constants and simulate pSTAT1 and pSTAT3 after 1 nM ligand.



Test the method on synthetic data

Notebook: Section 2

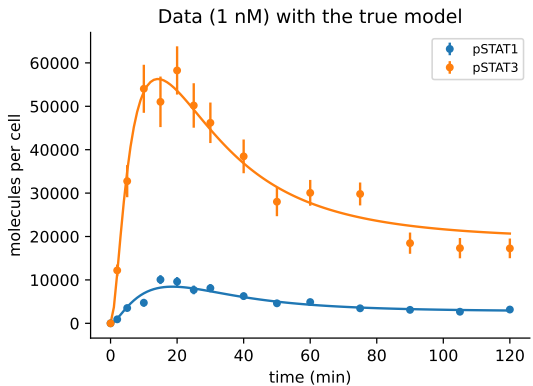
- Choose “true” rate constants and simulate pSTAT1 and pSTAT3 after 1 nM ligand.
- Sample 15 time points over 2 hours and add noise: SD = 10% of the value + 200 molecules.



Test the method on synthetic data

Notebook: Section 2

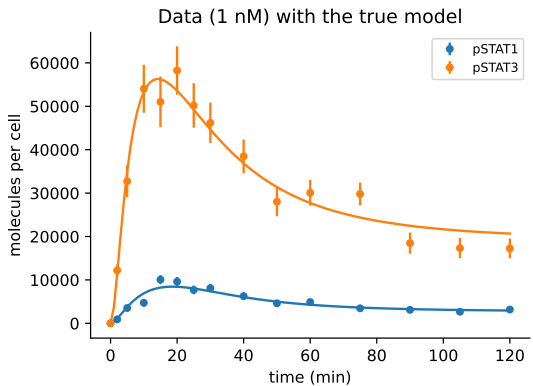
- Choose “true” rate constants and simulate pSTAT1 and pSTAT3 after 1 nM ligand.
- Sample 15 time points over 2 hours and add noise: SD = 10% of the value + 200 molecules.
- Then forget the true values, and see how many we can recover.



Test the method on synthetic data

Notebook: Section 2

- Choose “true” rate constants and simulate pSTAT1 and pSTAT3 after 1 nM ligand.
- Sample 15 time points over 2 hours and add noise: SD = 10% of the value + 200 molecules.
- Then forget the true values, and see how many we can recover.
- With real data there is no answer key. Synthetic data tell us what the method can and cannot do.



Part 2

Sensitivity analysis

Which parameters matter?

- A **parameter scan** varies one or two parameters over a range and follows the outputs. It maps out behaviors, but does not scale to 18 parameters.

Which parameters matter?

- A **parameter scan** varies one or two parameters over a range and follows the outputs. It maps out behaviors, but does not scale to 18 parameters.
- **Sensitivity analysis** measures the effect of every parameter on the outputs of interest.

Which parameters matter?

- A **parameter scan** varies one or two parameters over a range and follows the outputs. It maps out behaviors, but does not scale to 18 parameters.
- **Sensitivity analysis** measures the effect of every parameter on the outputs of interest.
- Goals: find the parameters with the **most** effect, and those with the **least**. Use this to guide estimation and experiments.

Which parameters matter?

- A **parameter scan** varies one or two parameters over a range and follows the outputs. It maps out behaviors, but does not scale to 18 parameters.
- **Sensitivity analysis** measures the effect of every parameter on the outputs of interest.
- Goals: find the parameters with the **most** effect, and those with the **least**. Use this to guide estimation and experiments.
- **Local**: small changes around one parameter set (derivatives).

Which parameters matter?

- A **parameter scan** varies one or two parameters over a range and follows the outputs. It maps out behaviors, but does not scale to 18 parameters.
- **Sensitivity analysis** measures the effect of every parameter on the outputs of interest.
- Goals: find the parameters with the **most** effect, and those with the **least**. Use this to guide estimation and experiments.
- **Local**: small changes around one parameter set (derivatives).
- **Global**: all parameters varied together over their plausible ranges.

Local sensitivity

Notebook: Section 3a

Sensitivity of output O_i to parameter p_j:

$$S_{ij}(t) = \frac{\partial \ln O_i(t)}{\partial \ln p_j} = \frac{p_j}{O_i}\frac{\partial O_i}{\partial p_j}$$

$S = 0.5$: a 1% increase in p_j raises O_i by 0.5%.
Independent of units.

Local sensitivity

Notebook: Section 3a

Sensitivity of output O_i to parameter p_j:

$$S_{ij}(t) = \frac{\partial \ln O_i(t)}{\partial \ln p_j} = \frac{p_j}{O_i}\frac{\partial O_i}{\partial p_j}$$

$S = 0.5$: a 1% increase in p_j raises O_i by 0.5%.
Independent of units.

Finite differences: rerun with $p_j \to (1+\epsilon)p_j$; one extra simulation per parameter, and ϵ must be chosen with care.

Local sensitivity

Notebook: Section 3a

Sensitivity of output O_i to parameter p_j:

$$S_{ij}(t) = \frac{\partial \ln O_i(t)}{\partial \ln p_j} = \frac{p_j}{O_i}\frac{\partial O_i}{\partial p_j}$$

$S = 0.5$: a 1% increase in p_j raises O_i by 0.5%. Independent of units.

Finite differences: rerun with $p_j \to (1+\epsilon)p_j$; one extra simulation per parameter, and ϵ must be chosen with care.

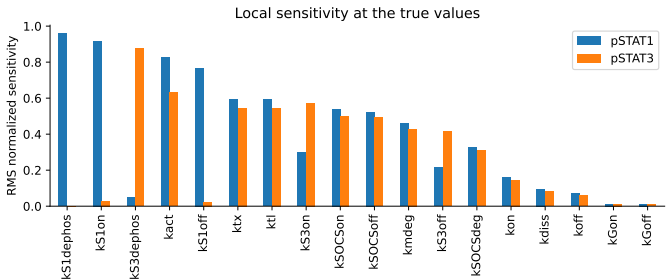
Forward sensitivity equations: integrate

$$\frac{d}{dt}\frac{\partial \mathbf{x}}{\partial p_j} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}}\frac{\partial \mathbf{x}}{\partial p_j} + \frac{\partial \mathbf{f}}{\partial p_j}$$

alongside $\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}; \mathbf{p})$. Exact, in one run: `Simulator(..., sensitivity_params=[...])` in bngsim.

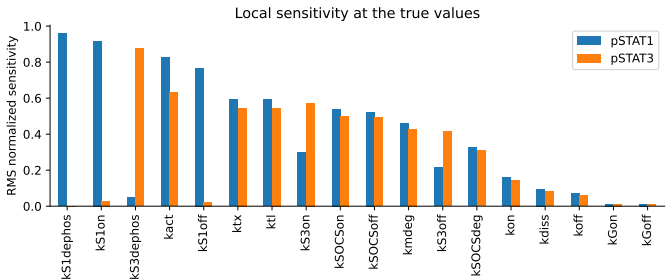
Local sensitivities of pSTAT1 and pSTAT3

Notebook: Section 3a



Local sensitivities of pSTAT1 and pSTAT3

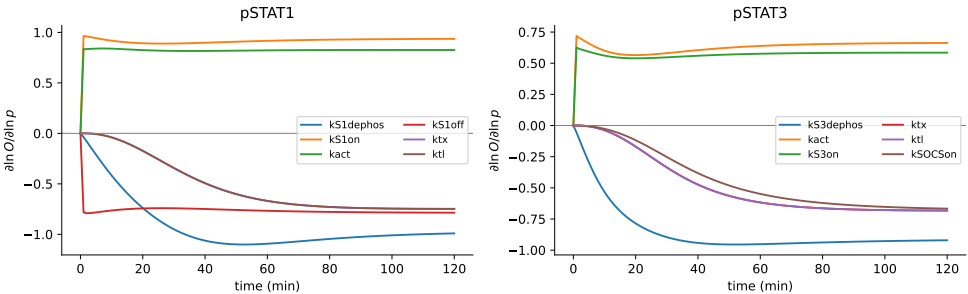
Notebook: Section 3a



- Each STAT's own dephosphorylation rate dominates its response.
- Docking and `kact` set the level. Competition: `kS3on` *lowers* pSTAT1 ($S \approx -0.3$).
- `ktx` and `ktl` are identical. Promoter binding and ligand binding barely matter: both are saturated.

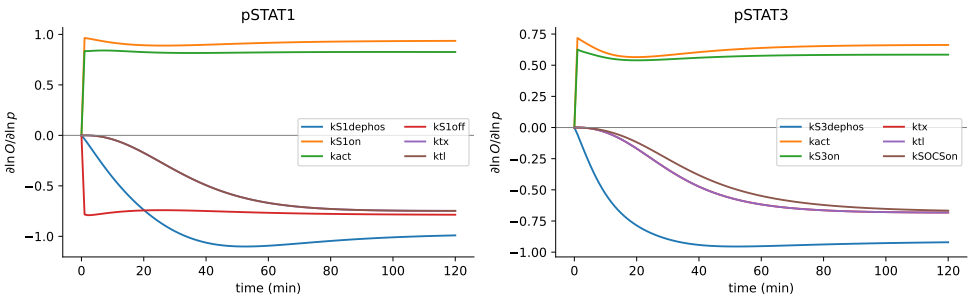
Sensitivities change over time

Notebook: Section 3a



Sensitivities change over time

Notebook: Section 3a



- Docking and kinase rates matter from the start; the SOCS3 loop (`ktx`, `ktl`, `kSOCSon`) only after about 20 min, once SOCS3 has accumulated.

Global sensitivity: sample the whole range

Notebook: Section 3b

1. Draw N parameter sets in $\log_{10}$ space by **Latin hypercube sampling**: each range is cut into N strata, each used once.

Global sensitivity: sample the whole range

Notebook: Section 3b

1. Draw N parameter sets in $\log_{10}$ space by **Latin hypercube sampling**: each range is cut into N strata, each used once.
2. Simulate each set and compute output features: peak, time of peak, value at 60 min.

Global sensitivity: sample the whole range

Notebook: Section 3b

1. Draw N parameter sets in $\log_{10}$ space by **Latin hypercube sampling**: each range is cut into N strata, each used once.
2. Simulate each set and compute output features: peak, time of peak, value at 60 min.
3. Regress each (log) feature on the (log) parameters, both standardized: $\tilde{O} \approx \sum_j \beta_j \tilde{p}_j$. The **standardized regression coefficients** β_j rank the parameters; R^2 says how much a linear model explains.

Global sensitivity: sample the whole range

Notebook: Section 3b

1. Draw N parameter sets in $\log_{10}$ space by **Latin hypercube sampling**: each range is cut into N strata, each used once.
2. Simulate each set and compute output features: peak, time of peak, value at 60 min.
3. Regress each (log) feature on the (log) parameters, both standardized: $\tilde{O} \approx \sum_j \beta_j \tilde{p}_j$. The **standardized regression coefficients** β_j rank the parameters; R^2 says how much a linear model explains.

Why log space?

Rate constants spread over orders of magnitude. Sampling uniformly in linear space puts 90% of the samples in the top decade and washes out the rest.

Global sensitivity: sample the whole range

Notebook: Section 3b

1. Draw N parameter sets in $\log_{10}$ space by **Latin hypercube sampling**: each range is cut into N strata, each used once.
2. Simulate each set and compute output features: peak, time of peak, value at 60 min.
3. Regress each (log) feature on the (log) parameters, both standardized: $\tilde{O} \approx \sum_j \beta_j \tilde{p}_j$. The **standardized regression coefficients** β_j rank the parameters; R^2 says how much a linear model explains.

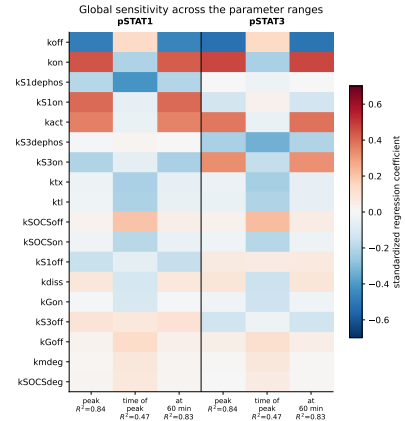
Why log space?

Rate constants spread over orders of magnitude. Sampling uniformly in linear space puts 90% of the samples in the top decade and washes out the rest.

2000 simulations of this model take a few seconds.

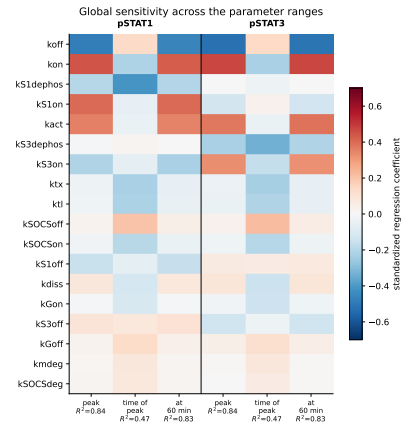
Global sensitivities

Notebook: Section 3b



Global sensitivities

Notebook: Section 3b



- **Ligand binding ranks first**, although it barely mattered locally. Across the ranges binding is often slow or weak, and then receptor occupancy limits everything.
- Dephosphorylation, docking, and `kact` matter in both analyses.
- Some locally important parameters (`kS1off`, the SOCS3 loop) average out globally.
- Peak and 60 min value: $R^2 \approx 0.84$. Time of peak: $R^2 \approx 0.47$, shaped by interactions a linear model misses.

Part 3

Identifiability

Sensitive is not the same as identifiable

- A parameter can strongly affect the outputs and still not be determined by the data, because other parameters can **compensate** for it.

Sensitive is not the same as identifiable

- A parameter can strongly affect the outputs and still not be determined by the data, because other parameters can **compensate** for it.
- **Structural** non-identifiability: no data of this kind, however good, can determine the parameter. In our model, the outputs depend on `ktx` and `ktl` only through their product: doubling one and halving the other changes nothing.

Sensitive is not the same as identifiable

- A parameter can strongly affect the outputs and still not be determined by the data, because other parameters can **compensate** for it.
- **Structural** non-identifiability: no data of this kind, however good, can determine the parameter. In our model, the outputs depend on `ktx` and `ktl` only through their product: doubling one and halving the other changes nothing.
- **Practical** non-identifiability: the parameter could be determined in principle, but these data, with this noise, do not constrain it.

Sensitive is not the same as identifiable

- A parameter can strongly affect the outputs and still not be determined by the data, because other parameters can **compensate** for it.
- **Structural** non-identifiability: no data of this kind, however good, can determine the parameter. In our model, the outputs depend on `ktx` and `ktl` only through their product: doubling one and halving the other changes nothing.
- **Practical** non-identifiability: the parameter could be determined in principle, but these data, with this noise, do not constrain it.
- Both kinds can be diagnosed from the model, before any fitting.

The Fisher information matrix

Notebook: Section 4

For data y_k with standard deviations σ_k:

$$\mathrm{FIM} = J^{\mathsf{T}} J, \qquad J_{kj} = \frac{1}{\sigma_k} \frac{\partial y_k}{\partial \log_{10} p_j}$$

The Fisher information matrix

Notebook: Section 4

For data y_k with standard deviations σ_k:

$$\mathrm{FIM} = J^{\mathsf{T}} J, \qquad J_{kj} = \frac{1}{\sigma_k} \frac{\partial y_k}{\partial \log_{10} p_j}$$

- Built from sensitivities at the data time points.
- Eigenvectors: **combinations** of parameters.
- Eigenvalue λ: how well the data constrain that combination. Standard error $\approx 1/\sqrt{\lambda}$ in $\log_{10}$ units.

The Fisher information matrix

Notebook: Section 4

For data y_k with standard deviations σ_k:

$$\mathrm{FIM} = J^{\mathsf{T}} J, \qquad J_{kj} = \frac{1}{\sigma_k} \frac{\partial y_k}{\partial \log_{10} p_j}$$

- Built from sensitivities at the data time points.
- Eigenvectors: **combinations** of parameters.
- Eigenvalue λ: how well the data constrain that combination. Standard error $\approx 1/\sqrt{\lambda}$ in $\log_{10}$ units.

λ	combination known to within
100	factor of 1.26
1	factor of 10
10^{-4}	not at all

The Fisher information matrix

Notebook: Section 4

For data y_k with standard deviations σ_k:

$$\mathrm{FIM} = J^{\mathsf{T}} J, \qquad J_{kj} = \frac{1}{\sigma_k} \frac{\partial y_k}{\partial \log_{10} p_j}$$

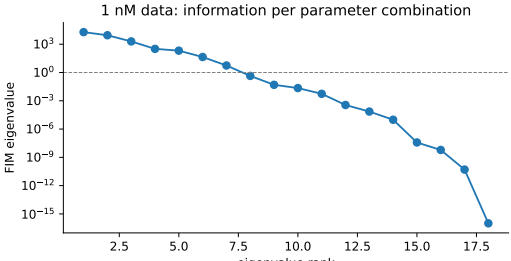
- Built from sensitivities at the data time points.
- Eigenvectors: **combinations** of parameters.
- Eigenvalue λ: how well the data constrain that combination. Standard error $\approx 1/\sqrt{\lambda}$ in $\log_{10}$ units.

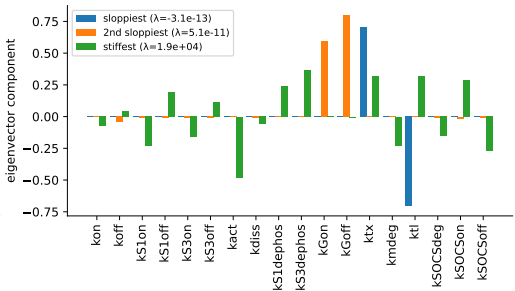
λ	combination known to within
100	factor of 1.26
1	factor of 10
10^{-4}	not at all

Needs no data, only the planned measurements and their noise, so it can evaluate an experiment **before** you do it.

Signaling models are sloppy

Notebook: Section 4

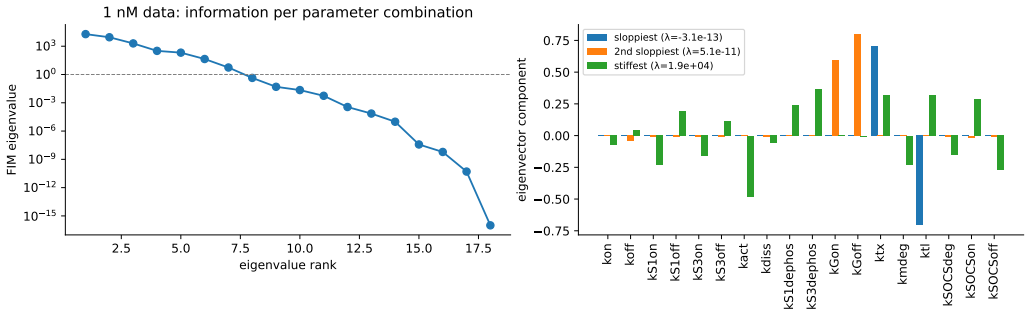




Gutenkunst et al., *PLoS Comput. Biol.* **3**, e189 (2007)

Signaling models are sloppy

Notebook: Section 4



- Eigenvalues span 17 decades; only 7 of 18 combinations exceed 1.
- Sloppiest: `ktx−ktl`, a **structural** non-identifiability (only the product matters). Next: promoter binding, which is saturated.

Gutenkunst et al., *PLoS Comput. Biol.* **3**, e189 (2007)

Part 4

Parameter estimation

Estimation as optimization

Find the parameters **p** that minimize a **cost function**.

Estimation as optimization

Find the parameters **p** that minimize a **cost function**.

Sum of squares: $C(\mathbf{p}) = \sum_k \left(y_k(\mathbf{p}) - y_k^{\text{obs}}\right)^2$

Estimation as optimization

Find the parameters **p** that minimize a **cost function**.

Sum of squares: $C(\mathbf{p}) = \sum_k \left(y_k(\mathbf{p}) - y_k^{\text{obs}}\right)^2$

With measurement errors σ_k:

$$\chi^2(\mathbf{p}) = \sum_k \left(\frac{y_k(\mathbf{p}) - y_k^{\text{obs}}}{\sigma_k}\right)^2 = -2 \ln P(\text{data} \mid \mathbf{p}) + \text{const}$$

for independent Gaussian errors. Minimizing χ^2 is maximizing the likelihood.

Estimation as optimization

Find the parameters **p** that minimize a **cost function**.

Sum of squares: $C(\mathbf{p}) = \sum_k \left(y_k(\mathbf{p}) - y_k^{\text{obs}}\right)^2$

With measurement errors σ_k:

$$\chi^2(\mathbf{p}) = \sum_k \left(\frac{y_k(\mathbf{p}) - y_k^{\text{obs}}}{\sigma_k}\right)^2 = -2 \ln P(\text{data} \mid \mathbf{p}) + \text{const}$$

for independent Gaussian errors. Minimizing χ^2 is maximizing the likelihood.

Is the fit good?

For a correct model, $\chi^2 \approx N - M$ (N data points, M fitted parameters). The probability of a larger χ^2 by chance is $Q = 1 - \Gamma\left(\frac{N-M}{2}, \frac{\chi^2}{2}\right)$.

Estimation as optimization

Find the parameters **p** that minimize a **cost function**.

Sum of squares: $C(\mathbf{p}) = \sum_k \left(y_k(\mathbf{p}) - y_k^{\text{obs}}\right)^2$

With measurement errors σ_k:

$$\chi^2(\mathbf{p}) = \sum_k \left(\frac{y_k(\mathbf{p}) - y_k^{\text{obs}}}{\sigma_k}\right)^2 = -2 \ln P(\text{data} \mid \mathbf{p}) + \text{const}$$

for independent Gaussian errors. Minimizing χ^2 is maximizing the likelihood.

Is the fit good?

For a correct model, $\chi^2 \approx N - M$ (N data points, M fitted parameters). The probability of a larger χ^2 by chance is $Q = 1 - \Gamma\left(\frac{N-M}{2}, \frac{\chi^2}{2}\right)$.

Meaningful only if the σ_k are reliable, which in systems biology they often are not.

Optimizers

Gradient-free

- Nelder–Mead simplex
- genetic algorithms, differential evolution
- particle swarm

Robust, but need many evaluations.

Optimizers

Gradient-free

- Nelder–Mead simplex
- genetic algorithms, differential evolution
- particle swarm

Robust, but need many evaluations.

Gradient-based

- steepest descent, conjugate gradient, L-BFGS
- Newton: fit a quadratic, step to its minimum
- for sums of squares: Gauss–Newton, Levenberg–Marquardt, trust region

Optimizers

Gradient-free

- Nelder–Mead simplex
- genetic algorithms, differential evolution
- particle swarm

Robust, but need many evaluations.

Gradient-based

- steepest descent, conjugate gradient, L-BFGS
- Newton: fit a quadratic, step to its minimum
- for sums of squares: Gauss–Newton, Levenberg–Marquardt, trust region

Gradients from sensitivities

For χ^2 the residuals are $r_k = (y_k - y_k^{\text{obs}})/\sigma_k$, and their Jacobian $\partial r_k / \partial p_j$ is the same scaled sensitivity matrix J as in the FIM. Least-squares methods use J to approximate the curvature: $\nabla^2 \chi^2 \approx 2J^{\mathsf{T}}J$.

Optimizers

Gradient-free

- Nelder–Mead simplex
- genetic algorithms, differential evolution
- particle swarm

Robust, but need many evaluations.

Gradient-based

- steepest descent, conjugate gradient, L-BFGS
- Newton: fit a quadratic, step to its minimum
- for sums of squares: Gauss–Newton, Levenberg–Marquardt, trust region

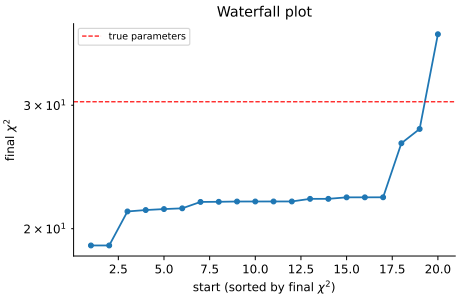
Gradients from sensitivities

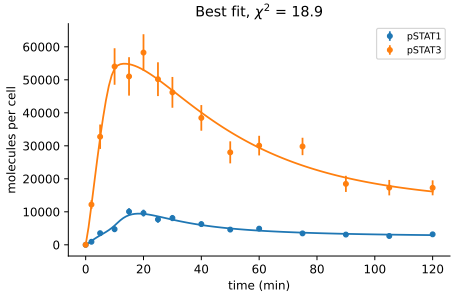
For χ^2 the residuals are $r_k = (y_k - y_k^{\text{obs}})/\sigma_k$, and their Jacobian $\partial r_k / \partial p_j$ is the same scaled sensitivity matrix J as in the FIM. Least-squares methods use J to approximate the curvature: $\nabla^2 \chi^2 \approx 2J^{\mathsf{T}}J$.

With forward sensitivities, each optimizer step costs about one simulation.

Multistart: the landscape has many minima

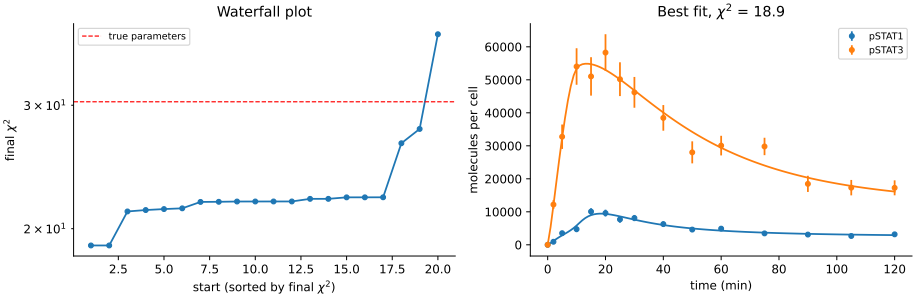
Notebook: Section 5





Multistart: the landscape has many minima

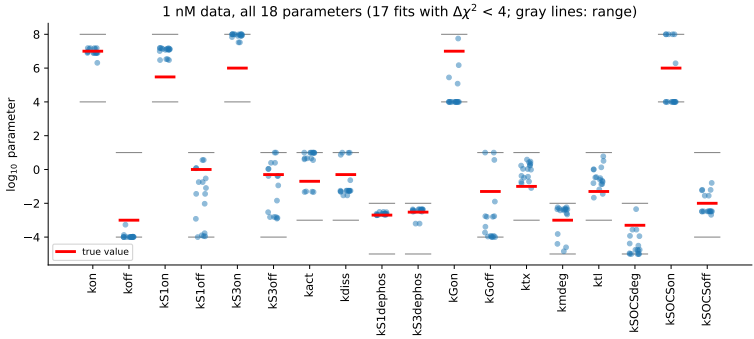
Notebook: Section 5



- 17 of 20 starts reach nearly the same χ^2. Best fit: χ^2 = 18.9, *below* the 30.3 of the true parameters: the fit follows the noise.

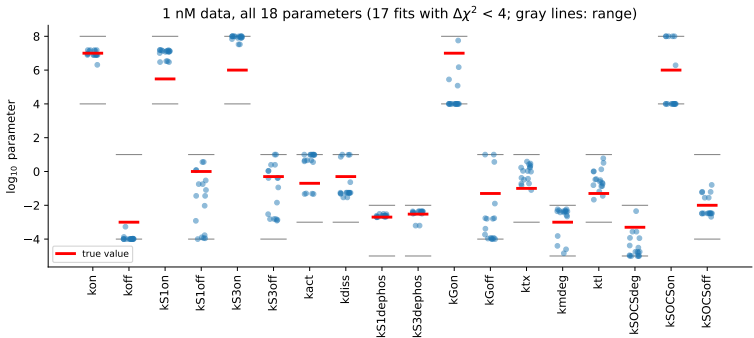
A good fit does not mean good estimates

Notebook: Section 5



A good fit does not mean good estimates

Notebook: Section 5



- Only `kS1dephos` is pinned down. `kS3on` and `koff` sit at bounds in every good fit; docking off-rates and the SOCS3 loop scatter.
- As the FIM predicted: about 7 combinations are constrained, not 18.

Profile likelihood

Notebook: Section 6

Fix p_i at a series of values; at each one, re-optimize everything else:

$$\mathrm{PL}(p_i) = \min_{p_{j \neq i}} \chi^2(\mathbf{p})$$

Profile likelihood

Notebook: Section 6

Fix p_i at a series of values; at each one, re-optimize everything else:

$$\mathrm{PL}(p_i) = \min_{p_{j \neq i}} \chi^2(\mathbf{p})$$

The 95% confidence interval is where PL stays within 3.84 of its minimum (χ^2 with one degree of freedom).

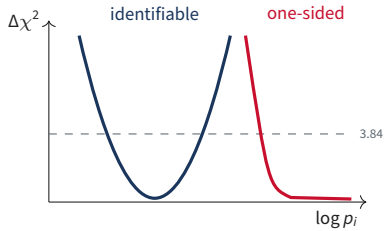
Profile likelihood

Notebook: Section 6

Fix p_i at a series of values; at each one, re-optimize everything else:

$$\mathrm{PL}(p_i) = \min_{p_{j \neq i}} \chi^2(\mathbf{p})$$

The 95% confidence interval is where PL stays within 3.84 of its minimum (χ^2 with one degree of freedom).

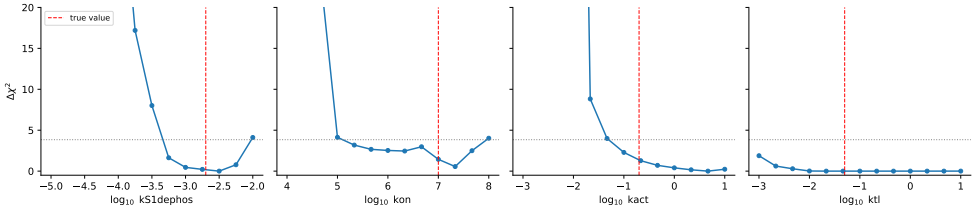


A profile that stays flat to a bound means the data put no limit on the parameter in that direction: the others compensate.

Raue et al., *Bioinformatics* **25**, 1923 (2009)

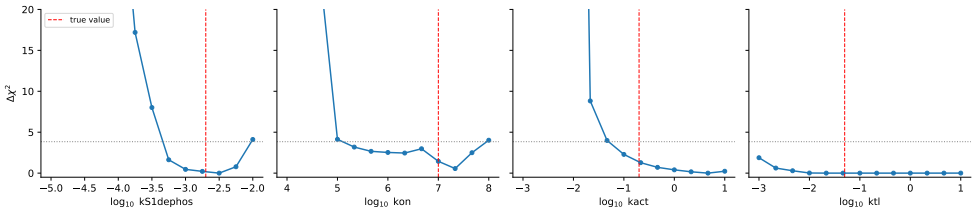
Profiles for four parameters

Notebook: Section 6



Profiles for four parameters

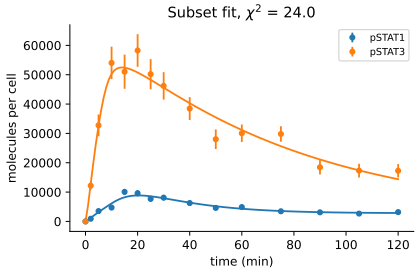
Notebook: Section 6

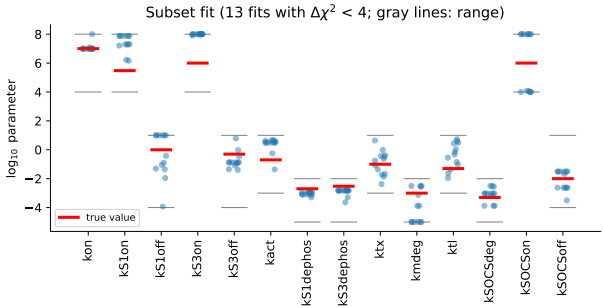


- `kS1dephos`: bounded on both sides, true value inside the 95% interval.
- `kon`, `kact`: bounded below only. Slow binding and a slow kinase are ruled out; faster values fit about as well.
- `ktl`: flat. `ktx` always compensates.

A tempting shortcut: fix the insensitive parameters

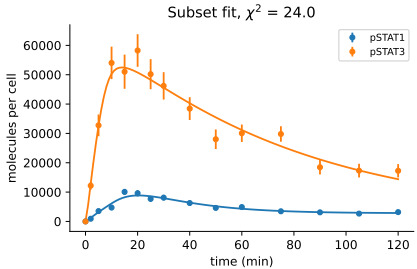
Notebook: Section 7

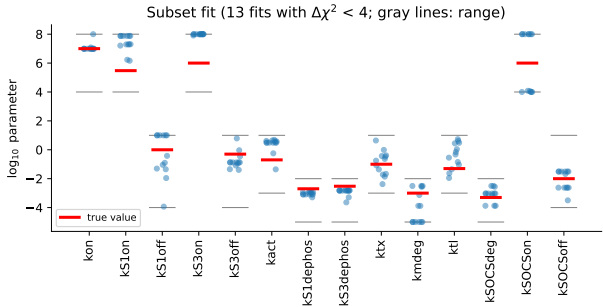




A tempting shortcut: fix the insensitive parameters

Notebook: Section 7

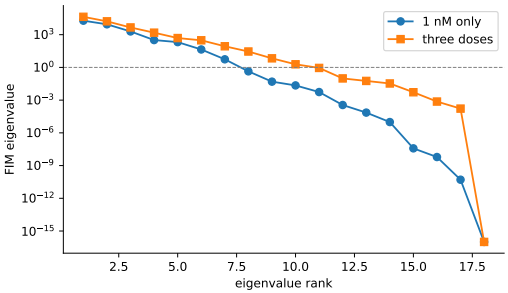


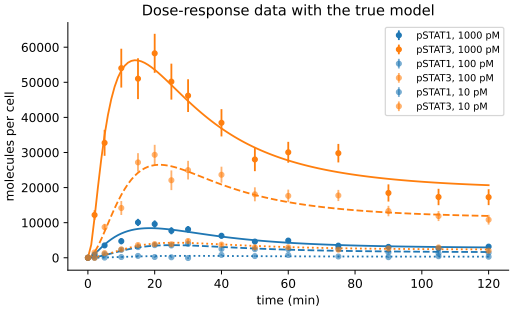


- Fixed `koff`, `kdiss`, `kGon`, `kGoff` (locally insensitive) at range centers. Fit almost as good: χ^2 = 24.0 vs. 18.9.
- But `koff` is now 30× too high, receptors are only ∼25% occupied, and other parameters (`kact`) compensate. Biased, with no warning.

Design the next experiment with the model

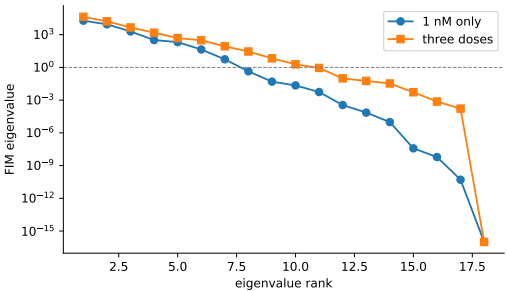
Notebook: Section 8

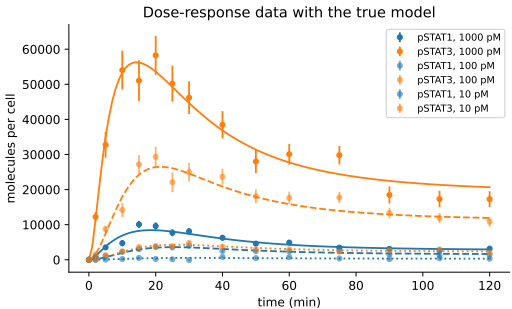




Design the next experiment with the model

Notebook: Section 8

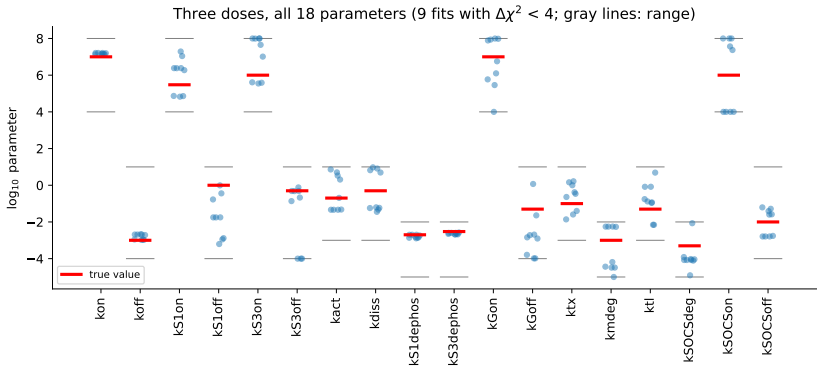




- Add 100 pM and 10 pM doses: eigenvalues above 1 go from 7 to 10, and the sloppy directions gain several decades. `ktx`/`ktl` stays at zero.

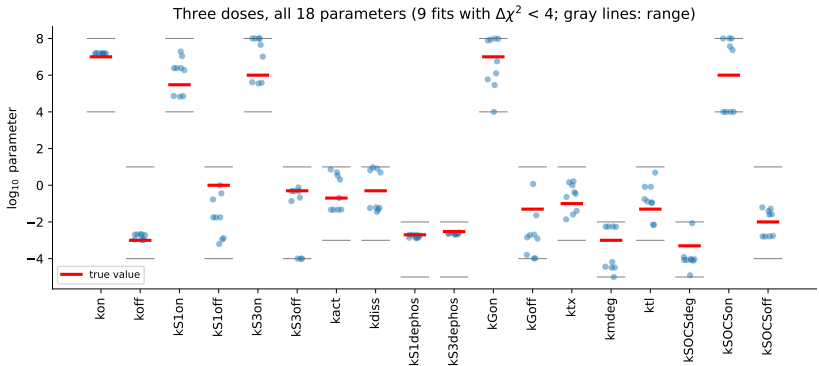
More informative data, better estimates

Notebook: Section 8



More informative data, better estimates

Notebook: Section 8



- `koff` moves from its bound to its true value; both dephosphorylation rates are pinned. The docking cycle is still undetermined: the next experiment should measure closer to it (receptor-bound STAT).

Try it yourself

Work through `notebooks/stat1_stat3_fitting.ipynb`, then:

1. Change the noise to 5% and to 20%. How do the FIM eigenvalues and the spread of good fits change?
2. Replace `ktx` and `ktl` by their product. Is the product identifiable?
3. Add total SOCS3 protein as a measured output. Which parameters become identifiable?
4. Predict a SOCS3 knockout (`G0 = 0`) with each good fit. Do the fits agree? What does this say about predictions from a sloppy model?



The notebook

Summary

- Rate constants come in standard units; models of single cells often count **molecules**. Convert with $N_A V$, using the volume where the reaction happens.

Summary

- Rate constants come in standard units; models of single cells often count **molecules**. Convert with $N_A V$, using the volume where the reaction happens.
- **Sensitivity analysis** finds the parameters that move the outputs: locally with derivatives, globally by sampling the plausible ranges.

Summary

- Rate constants come in standard units; models of single cells often count **molecules**. Convert with $N_A V$, using the volume where the reaction happens.
- **Sensitivity analysis** finds the parameters that move the outputs: locally with derivatives, globally by sampling the plausible ranges.
- **Identifiability** concerns combinations of parameters. The FIM predicts which ones the data constrain, before fitting or even measuring.

Summary

- Rate constants come in standard units; models of single cells often count **molecules**. Convert with $N_A V$, using the volume where the reaction happens.
- **Sensitivity analysis** finds the parameters that move the outputs: locally with derivatives, globally by sampling the plausible ranges.
- **Identifiability** concerns combinations of parameters. The FIM predicts which ones the data constrain, before fitting or even measuring.
- **Multistart fitting** finds the best fit, but many parameter sets fit equally well. Check **profiles** before trusting an estimate.

Summary

- Rate constants come in standard units; models of single cells often count **molecules**. Convert with $N_A V$, using the volume where the reaction happens.
- **Sensitivity analysis** finds the parameters that move the outputs: locally with derivatives, globally by sampling the plausible ranges.
- **Identifiability** concerns combinations of parameters. The FIM predicts which ones the data constrain, before fitting or even measuring.
- **Multistart fitting** finds the best fit, but many parameter sets fit equally well. Check **profiles** before trusting an estimate.
- Use the model to **design the next experiment**.

Additional reading

- R. N. Gutenkunst et al. (2007). **Universally sloppy parameter sensitivities in systems biology models.** *PLoS Comput. Biol.* **3**, e189.
- A. Raue et al. (2009). **Structural and practical identifiability analysis of partially observed dynamical models by exploiting the profile likelihood.** *Bioinformatics* **25**, 1923–1929.
- S. Marino et al. (2008). **A methodology for performing global uncertainty and sensitivity analysis in systems biology.** *J. Theor. Biol.* **254**, 178–196.
- E. D. Mitra and W. S. Hlavacek (2019). **Parameter estimation and uncertainty quantification for systems biology models.** *Curr. Opin. Syst. Biol.* **18**, 9–18.
- Y. Schälte et al. (2023). **pyPESTO: a modular and scalable tool for parameter estimation for dynamic models.** *Bioinformatics* **39**, btad711.